



TECLADO E SOLUÇÕES OEM

AVANTTEC TECNOLOGIA... EM TECLADOS

Somos uma empresa especializada em soluções personalizadas para **automação comercial bancária e industrial**, com eficiência e qualidade. Nosso compromisso é atender à expectativa dos nossos parceiros na **fabricação e assistência técnica de teclados** de qualquer marca e modelo.

REFERÊNCIA DE API **LIBK044AVT.SO** Teclado 44 Teclas PS/2 TEC44AVT



EMPRESA



Avanttec Máquinas e Equipamentos Industriais
Rua Puroborás , 61 – Vila Conceição

Diadema - SP

CEP: 09.990-340

TELEFONE



Tel.: +55 (11) 9 9185-3191

WHATSAPP



Tel.: +55 (11) 9 9185-3191

EMAIL



suporte@avanttectecnologia.com.br
vendas@avanttectecnologia.com.br
projetos@avanttectecnologia.com.br
financeiro@avanttectecnologia.com.br
compras@avanttectecnologia.com.br

Índice

Índice	1
1.Introdução.....	2
2.Rotinas de Ciclo de Vida.....	2
Estrutura k044_config_t.....	3
3.Rotinas de Informações do Dispositivo e da Biblioteca	4
4.Rotinas de comandos PS/2 padrão	4
5.Rotinas de Tratamento do DISPLAY (Read/Write)	5
6.Rotinas de Tratamento do DISPLAY (Cursor)	6
7.Rotinas do Dispositivo (Avanttec TEC44FST Keyboard-Mouse).....	7
8.Iniciando os Dispositivos e Lendo os seus Eventos	8
Cenários Típicos de Integração.....	8
Regra Geral sobre o Event Loop	9
9.Leitura Assíncrona e Síncrona (Event Loop)	10
Tipos de Eventos Disponíveis (k044_event_type_t)	10
10.Exibição de Teclas no Display (KeyEcho)	11
Estrutura k044_keyecho_cfg_t	11
11.Modulo de Impressão Digital (FingerPrint).....	12
12.Termo de Serviço.....	16
13.Termo de Assistência Técnica.....	16

1. Introdução

A biblioteca **libK044AVT.so** foi desenvolvida e projetada para simplificar a integração e o controle do teclado **TEC44AVT** da **Avanttec Tecnologia** em suas aplicações, abstraindo a complexidade do barramento PS/2 e fornecendo uma interface de programação (API) intuitiva.

Códigos de retorno

Todas as rotinas retornam um int com um dos códigos abaixo, salvo indicação em contrário. K044_OK (0) sempre indica sucesso.

K044_OK	0	Sucesso
K044_ERR_PERM	-1	Permissão negada (requer root ou setcap CAP_SYS_RAWIO)
K044_ERR_TIMEOUT	-2	Dispositivo não respondeu no prazo
K044_ERR_NODEV	-3	Dispositivo não encontrado / ID inválido
K044_ERR_RANGE	-4	Parâmetro fora do intervalo válido
K044_ERR_INVALID	-5	Parâmetro inválido
K044_ERR_BUSY	-6	Recurso ocupado (ex.: event loop está ativo)
K044_ERR_RESEND	-7	Dispositivo solicitou retransmissão
K044_ERR_FP	-8	Erro do módulo fingerprint
K044_ERR_FP_TIMEOUT	-9	Timeout aguardando o módulo fingerprint

2. Rotinas de Ciclo de Vida

k044_open(void)

Abre o dispositivo com configuração padrão.

Retorno: K044_OK ou erro

k044_open_ex(const k044_config t *cfg)

Abre o dispositivo com configuração personalizada. cfg=NULL usa padrão.

Retorno: K044_OK ou erro

k044_close(void)

Fecha o dispositivo.

Retorno: void

k044_lock(void)

Captura o mutex do barramento PS/2.

Retorno: K044_OK

k044_unlock(void)

Libera o mutex capturado por k044_lock().

Retorno: void

k044_open() vs k044_open_ex()

k044_open() abre o dispositivo com a configuração padrão (portas PS/2 0x60/0x64, timeouts padrão)

k044_open_ex() abre com uma *k044_config_t* personalizada, útil quando você precisa mudar portas, timeouts ou nível de log. Chame **k044_config_init()** para preencher os padrões e só altere o campo que precisar; **k044_open_ex(NULL)** equivale a **k044_open()**.

Para que servem: k044_lock() — k044_unlock()

Garantem atomicidade quando você precisa fazer várias chamadas seguidas sem que outra thread (ex.: a do event loop) intercale uma transação PS/2 no meio.

A maioria dos programas nunca precisa chamá-los diretamente, pois as próprias rotinas da biblioteca já usam o lock internamente quando fazem uma transação composta.

Estrutura k044_config_t

```
typedef struct {
    uint16_t port_data;           // porta de dados PS/2      (padrão: 0x60)
    uint16_t port_status;        // porta de status PS/2    (padrão: 0x64)
    uint32_t write_timeout_ms;    // timeout de escrita, ms  (padrão: 100)
    uint32_t read_timeout_ms;     // timeout de leitura, ms  (padrão: 200)
    uint32_t reset_wait_ms;       // espera após reset, ms   (padrão: 1000)
    int      max_retries;         // tentativas em RESEND    (padrão: 3)
    int      log_level;           // nível de log             (padrão: NONE)
} k044_config_t;
```

CAMPO	PADRÃO	DESCRIÇÃO
port_data	0x60	Porta de dados do barramento PS/2
port_status	0x64	Porta de status do barramento PS/2
write_timeout_ms	100 ms	Espera por confirmação ao enviar um comando
read_timeout_ms	200 ms	Espera por uma resposta/byte do dispositivo
reset_wait_ms	1000 ms	Espera após o comando de reset (0xFF)
max_retries	3	Retransmissões automáticas em K044_ERR_RESEND
log_level	LOG_NONE	Verbosidade dos logs internos (NONE/ERROR/WARN/DEBUG/TRACE)

Na prática

Os campos mais ajustados são **read_timeout_ms/write_timeout_ms** (hardware mais lento que o normal, ex.: junto com o sensor de digital) e **log_level** (para depurar problemas de comunicação); os demais quase nunca precisam mudar do padrão.

3. Rotinas de Informações do Dispositivo e da Biblioteca

k044_version(void)

Retorna a string de versao da biblioteca (ex.: "1.0.3").

Retorno: const char*

k044_read_id(uint8_t id[2])

Le o identificador do dispositivo (comando 0xF2). Sucesso: id={0xAB,0x83}.

Retorno: K044_OK ou erro

k044_firmware_version(char *buf, size_t len)

Le a versao do firmware como string (comando 0xF1), ex.: "K044P001A".

len minimo recomendado: 16.

Retorno: K044_OK ou erro

k044_read_fw_ver(uint8_t *major, uint8_t *minor, uint8_t *patch)

Le a versao numerica do firmware (comando 0xAF). Os ponteiros NULL.

Retorno: K044_OK, ou K044_ERR_NODEV se o firmware nao suportar comando 0xAF

k044_fw_ver_str(char *buf, size_t len)Formata "major.minor.patch" a partir do ultimo **k044_read_fw_ver()** bem sucedido. len minimo: 12.

Retorno: K044_OK ou erro

k044_read_serial(char *buf, size_t len)

Le o numero de serie do equipamento, 7 chars ASCII. len minimo: 8.

Retorno: K044_OK, ou K044_ERR_NODEV se o firmware nao suportar o comando

4. Rotinas de comandos PS/2 padrão

k044_reset(void)

Reinicia o firmware do dispositivo (comando 0xFF).

Retorno: K044_OK ou erro

k044_set_leds(uint8_t leds)

Controla os LEDs do teclado (comando 0xED). leds: bitmask de

K044_LED_SCROLLLOCK | K044_LED_NUMLOCK | K044_LED_CAPSLOCK.

Retorno: K044_OK ou erro

k044_set_typematic(uint8_t rate)

Define a taxa de repetição de teclas (comando 0xF3, encoding PS/2 padrão).

Retorno: K044_OK ou erro

5. Rotinas de Tratamento do DISPLAY (Read/Write)

k044_write_cgram_preset(uint8_t pattern, uint8_t slot)

Grava um padrão pré-definido (K044_CGRAM_PRESET_*) em um slot da CGRAM (0-7)

Retorno: K044_OK ou erro

k044_write_char(uint8_t c)

Escreve 1 byte na posição atual do cursor.

Bytes < 0x20 são interpretados como comandos de controle (K044_BS, K044_CR, etc.).

Retorno: K044_OK ou erro

k044_write_char_raw(uint8_t c)

Escreve 1 byte direto na DDRAM, sem interpretar códigos de controle

— Usar para exibir caracteres CGRAM (códigos 0-7).

Retorno: K044_OK ou erro

k044_write_string(const char *s)

Escreve string terminada em '\0' (max 80 chars, duas linhas).

Retorno: K044_OK ou erro

k044_write_buf(const uint8_t *buf, size_t len)

Escreve um buffer de bytes no display.

Retorno: K044_OK ou erro

k044_write_line(uint8_t row, const char *text)

Escreve uma linha inteira (0 ou 1), completando com espaços até a col 39.

Retorno: K044_OK ou erro

k044_display_shift(int direction)

Desloca o display inteiro (instrução nativa do controlador HD44780)

Direção: [0] = esquerda, [!= 0] = direita.

Retorno: K044_OK ou erro

k044_write_display(const char *line1, const char *line2)

Limpa o display e escreve as duas linhas de uma vez. NULL = linha vazia.

Retorno: K044_OK ou erro

k044_write_pos(uint8_t row, uint8_t col, const char *fmt, ...)

Escreve string formatada na posição (row, col). row: 0-1 | col: 0-39 | fmt/...: (estilo printf.)

Retorno: K044_OK ou erro

k044_scroll_line(uint8_t row, const char *text, unsigned int delay_ms)

Rola texto horizontalmente numa linha, um loop, janela deslizante.

Retorno: K044_OK ou erro

k044_scroll_start(uint8_t row, uint8_t col_start, uint8_t width, const char *text, unsigned int delay_ms, int repeat)

Inicia scroll contínuo em uma janela (col_start, width) da linha.

repeat: 0 = loop infinito, N = número de loops.

Retorno: K044_OK ou erro

k044_scroll_stop(void)

Para o scroll contínuo iniciado por **k044_scroll_start()**.

Retorno: K044_OK ou erro

6. Rotinas de Tratamento do DISPLAY (Cursor)

k044_clear(void)

Limpa o display inteiro.

Retorno: K044_OK/erro

k044_wait_busy(void)

Aguarda o dispositivo ficar livre.

Retorno: K044_OK, ou K044_ERR_TIMEOUT

k044_home(void)

Cursor para (0,0).

Retorno: K044_OK/erro

k044_set_cursor(uint8 t row, uint8 t col)

Posiciona cursor. row: 0-1 | col: 0-39.

Retorno: K044_OK, ou K044_ERR_RANGE se fora da faixa

k044_cursor_on(void)

Ativa exibição do cursor.

Retorno: K044_OK/erro

k044_cursor_off(void)

Oculto o cursor.

Retorno: K044_OK/erro

k044_carriage_return(void)

Cursor para coluna 0 da linha atual.

Retorno: K044_OK/erro

k044_line_feed(void)

Cursor para a linha 2.

Retorno: K044_OK/erro

k044_cursor_up(void)

Cursor para a linha 1.

Retorno: K044_OK/erro

k044_backspace(void)

Cursor uma posição a esquerda (apaga).

Retorno: K044_OK/erro

k044_tab(void)

Avança cursor uma posição.

Retorno: K044_OK/erro

k044_cursor_inc(void)

Avança cursor sem apagar (= tab).

Retorno: K044_OK/erro

k044_cursor_dec(void)

Recua cursor sem apagar.

Retorno: K044_OK/erro

k044_erase_eol(void)

Apaga da posição atual ate fim da linha.

Retorno: K044_OK/erro

k044_bell(void)

Sinal sonoro (sem efeito neste hardware).

Retorno: K044_OK/erro

7. Rotinas do Dispositivo (Avanttec TEC44FST Keyboard-Mouse)

k044_uinput_enable(void)

Cria um teclado virtual (/dev/uinput).

Retorno: K044_OK, K044_ERR_NODEV (uinput inacessível) ou K044_ERR_BUSY

k044_uinput_disable(void)

Desativa o teclado virtual.

Retorno: K044_OK ou erro

k044_uinput_is_active(void)

Verifica se o teclado virtual (uinput) está ativo no momento.

Retorno: 1 se ativo, 0 caso contrário

k044_mouse_enable(void)

Cria um mouse virtual (/dev/uinput) para repassar ao sistema os eventos de movimento e clique.

Retorno: K044_OK, K044_ERR_NODEV ou K044_ERR_BUSY

k044_mouse_disable(void)

Desativa o mouse virtual.

Retorno: K044_OK ou erro

k044_mouse_is_active(void)

Verifica se o mouse virtual está ativo no momento.

Retorno: 1 se ativo, 0 caso contrário

k044_set_mouse_auto(int enable)

Se enable=1 (chamar ANTES de k044_open()), ativa mouse+event_loop após k044_open().

Retorno: void

k044_is_device_dead(void)

Verifica se o dispositivo foi marcado inoperante após falha irreversível de protocolo.

Quando 1, toda E/S retorna K044_ERR_NODEV imediatamente.

Retorno: 0 = operacional, 1 = inoperante

k044_aux_enable(void)

Habilita o teclado auxiliar (PS/2, 110 teclas) conectado ao dispositivo.

ATENCAO: So utilize a rotina com o teclado auxiliar fisicamente conectado!

O sistema pode travar aguardando a resposta.

Retorno: K044_OK ou erro

k044_aux_disable(void)

Desabilita o teclado auxiliar.

Retorno: K044_OK ou erro

8. Iniciando os Dispositivos e Lendo os seus Eventos

Esta seção orienta, de forma direta, como combinar as rotinas da biblioteca de acordo com a arquitetura e os requisitos do seu aplicativo. Abaixo são apresentados a ordem correta das chamadas, os cenários típicos de uso e a escolha do modo de leitura ideal.

Cenários Típicos de Integração

1. Apenas Display/Cursor (Teclado inativo no sistema)

Objetivo:

Utilizar apenas a interface visual, sem que o teclado envie comandos para o sistema operacional.

Fluxo de Chamadas:

k044_open() seguido estritamente pelas rotinas de controle de display/cursor.

Não inicialize uinput, mouse ou dispositivos auxiliares (aux).

2. Terminal Interativo / Menu (Teclado ativo e funcional)

Objetivo:

Permitir que o usuário digite e navegue normalmente pelo sistema enquanto o software roda em segundo plano.

Fluxo de Chamadas:

Logo após o k044_open(), chame a sequência:

k044_uinput_enable() → k044_mouse_enable() → k044_start_event_loop() → k044_aux_enable()
(caso haja teclado auxiliar físico conectado).

3. Teclado Bloqueado (Aguardando validação/condição)

Objetivo:

Reter a digitação (ex.: travar o teclado até que uma digital seja reconhecida).

Fluxo de Chamadas:

Execute apenas k044_open(). As teclas serão capturadas internamente pela biblioteca, mas não serão injetadas no sistema operacional. Após a validação da condição, libere o fluxo invocando as funções de inicialização (uinput, mouse, aux e event_loop).

4. Eco de Teclas no Display (Campos de texto)

Objetivo:

Mostrar os caracteres digitados diretamente na tela de forma otimizada.

Fluxo de Chamadas:

Invoque diretamente k044_keyecho_start(). Esta função inicializa o laço de eventos (event loop) de forma autônoma se ele ainda não estiver em execução, dispensando a chamada manual de k044_start_event_loop().

5. Ativação Automática do Mouse PS/2

Objetivo:

Garantir o funcionamento imediato e automático do mouse desde a inicialização do software.

Fluxo de Chamadas:

Chame obrigatoriamente k044_set_mouse_auto(1) antes de executar o k044_open().

Esse procedimento dispensa a ativação manual posterior via

k044_mouse_enable() ou k044_start_event_loop().

Regra Geral sobre o Event Loop

A execução do **k044_start_event_loop()** é obrigatória para habilitar os seguintes recursos:

- ☒ Injeção das teclas via subsistema uinput;
- ☒ Ativação da leitura síncrona (**k044_read_event()**) ou assíncrona (**k044_set_event_callback()**);
- ☒ Ativação da rotina de eco em tela (**keyecho**).

Atenção

As funções **k044_read_event()** e **k044_keyecho_start()** gerenciam e iniciam o event loop sozinhas se necessário. Contudo, a função **k044_uinput_enable()** **NÃO** o inicia.

Se você habilitar o **uinput** sem disparar o **event loop**, as teclas não serão repassada ao sistema operacional.

Modos de Leitura de Evento — Qual Usar?

MODO	FUNÇÃO BASE	QUANDO UTILIZAR
Callback (Assíncrono)	k044_set_event_callback	Aplicações com interface gráfica (GUI) ou que possuam um laço de repetição próprio que não pode ser bloqueado aguardando a interação do usuário.
Síncrono (Bloqueante)	k044_read_event	Programas de terminal baseados em fluxos sequenciais simples (onde o sistema necessariamente espera uma tecla, processa o dado e aguarda a próxima ação).
Keyecho (Automatizado)	k044_keyecho_start	Quando o único objetivo é imprimir o que foi digitado no display. A biblioteca gerencia a escrita de forma autônoma e rápida, eliminando a necessidade de criar códigos manuais de tratamento de eventos.

Atenção ao Desempenho (Gargalo do Barramento)

Evite chamar rotinas manuais de display (como **k044_write_pos**, etc.) dentro de um loop de captura de eventos gerenciado por **k044_read_event()** ou pela função de **callback**.

Como as operações de escrita no visor e do teclado compartilham o mesmo barramento PS/2, cada transação de tela concorrente causará atrasos perceptíveis na digitação do usuário.

Quando precisar exibir caracteres digitados no display, use sempre a função nativa **k044_keyecho_start()**, que implementa uma via de escrita otimizada para mitigar esse problema.

9. Leitura Assíncrona e Síncrona (Event Loop)

O monitoramento de eventos permite que o programa seja notificado sobre ações dos dispositivos sem a necessidade de consulta constante (polling).

Existem dois modelos de implementação, escolha conforme a arquitetura do seu aplicativo.

1. Modelo Assíncrono (Callback)

Função: k044_set_event_callback + k044_start_event_loop

Funcionamento:

O evento é disparado automaticamente em uma thread interna dedicada sempre que uma nova ação ocorre.

Quando usar:

Ideal para aplicações com interface gráfica (GUI) ou que possuam um loop principal próprio que não pode ser bloqueado.

Exemplo:

Um sistema de PDV (Frente de Caixa) que precisa continuar processando comandos na tela enquanto aguarda o cliente passar o cartão.

Atenção:

A função de callback deve ser leve, rápida e thread-safe (evite operações pesadas de I/O ou bloqueios).

2. Modelo de Leitura Síncrona (Bloqueante)

Função: k044_read_event

Funcionamento:

O programa executa um loop simples que bloqueia a execução até que o próximo evento ocorra ou o tempo limite (timeout) seja atingido.

Quando usar:

Recomendado para aplicações de terminal (CLI) com fluxo sequencial, no estilo "espera o evento — processa — espera o próximo".

É um modelo mais simples e linear de codificar do que o callback.

NOTA DE ENCERRAMENTO

Sempre chame a função **k044_stop_event_loop()** quando não for mais necessário monitorar os periféricos, para liberar os recursos do sistema.

Tipos de Eventos Disponíveis (k044_event_type_t)

CATEGORIA	EVENTO(S)	DESCRIÇÃO
Teclado	K044_EVT_KEY_MAKE K044_EVT_KEY_BREAK	Tecla pressionada Tecla solta
Teclado PIN	K044_EVT_PIN_KEY	Dígito do PIN inserido.
Leitor de Cartão Magnético	K044_EVT_CARD_START K044_EVT_CARD_DATA K044_EVT_CARD_END K044_EVT_CARD_ERROR	Eventos da leitura de cartão magnético.
Dados Puros	K044_EVT_RAW	Byte bruto do barramento PS/2

10. Exibição de Teclas no Display (KeyEcho)

k044_keyecho_cfg_init(k044_keyecho_cfg_t *cfg)

Preenche **cfg** com os valores padrões (row=0, max_chars=32, confirm=Enter, etc.).

Retorno: void

k044_keyecho_start(const k044_keyecho_cfg_t *cfg, k044_keyecho_cb_t cb, void *userdata)

Inicia a exibição de teclas pressionadas no display (só o teclado 44 teclas).

cfg=NULL usa padrão. cb=NULL = sem notificação.

Retorno: K044_OK, ou K044_ERR_RANGE se a config for inválida,

k044_keyecho_stop(void)

Para o keyecho e apaga a área usada no display.

Retorno: K044_OK ou erro

k044_keyecho_get_buffer(char *buf, uint8_t *len)

Copia o conteúdo atual do buffer para buf (min. 16 bytes).

Retorno: K044_OK, ou K044_ERR_INVALID se buf/len forem NULL

k044_keyecho_get_cursor(uint8_t *pos)

Retorna a posição do cursor dentro do buffer (0-based).

Retorno: K044_OK, ou K044_ERR_INVALID se pos for NULL

k044_keyecho_clear(void)

Limpa o buffer.

Retorno: K044_OK ou erro

Estrutura k044_keyecho_cfg_t

```
typedef struct {
    uint8_t row;                // linha do display: 0 ou 1      (padrão: 0)
    uint8_t col_start;          // coluna inicial: 0-14        (padrão: 0)
    uint8_t max_chars;          // máx. chars visíveis: 1-40   (padrão: 32)
    uint16_t clear_scancode;     // tecla que limpa o buffer     (0=desativado)
    uint16_t confirm_scancode;   // tecla que confirma entrada   (padrão: 0x5A)
    int pin_mode;               // 1 = mostra '*' no lugar     (padrão: 0)
    int echo_enabled;           // 1 = escreve no display       (padrão: 1)
} k044_keyecho_cfg_t;
```

CAMPO	PADRÃO	DESCRIÇÃO
row / col_start	0 / 0	Onde no display o texto aparece
max_chars	32	Até onde o texto pode crescer (col_start + max_chars ≤ 40)
clear_scancode	0	Tecla que limpa o buffer inteiro (0 = desativado)
confirm_scancode	0x5A	Tecla que confirma a entrada (dispara CONFIRMED)
pin_mode	0	1 = mostra '*' em vez do caractere real
echo_enabled	1	0 = mantém buffer/callback sem escrever no display

Eventos k044_keyecho_event_t

EVENTO	DESCRIÇÃO
K044_KEYECHO_CHAR	Uma tecla normal foi digitada e adicionada ao buffer
K044_KEYECHO_CLEARED	A tecla de limpeza (clear_scancode) foi pressionada
K044_KEYECHO_BACKSPACE	Um caractere foi apagado à esquerda do cursor
K044_KEYECHO_DELETE	Um caractere foi apagado na posição atual do cursor
K044_KEYECHO_CONFIRMED	Tecla de confirmação pressionada — leia o buffer aqui
K044_KEYECHO_FULL	O buffer atingiu max_chars e a tecla foi ignorada

Na prática

A maioria dos programas só precisa tratar K044_KEYECHO_CONFIRMED (para pegar o resultado final) e talvez K044_KEYECHO_FULL (para dar feedback de limite excedido);

Os demais eventos existem para casos mais avançados de sincronização de estado.

11. Modulo de Impressão Digital (FingerPrint)

k044_fp_init(void)

Inicializa a comunicação com o modulo fingerprint. Não requer configuração adicional; chame uma vez após k044_open(), antes de qualquer outra rotina k044_fp_*.

Retorno: K044_OK ou erro

k044_fp_get_image(void)

Captura a imagem da digital no momento da chamada.

Primeiro passo de um cadastro/identificação manual, chamada antes de k044_fp_gen_char().

Retorno: K044_OK, ou K044_ERR_FP se sensor vazio/erro

k044_fp_gen_char(uint8_t buffer_id)

Gera o template (assinatura da digital) a partir da ultima imagem de k044_fp_get_image(), guardando no CharBuffer (memória temporária do sensor) indicado por [buffer_id](#).

Um cadastro manual utiliza os dois buffers antes de mescla-los com k044_fp_reg_model().

Retorno: K044_OK ou erro

k044_fp_match(void)

Compara os templates armazenados nos CharBuffers 1 e 2 verificando se pertencem a mesma digital.

Retorno: K044_OK se coincidem, K044_ERR_FP se não

k044_fp_search(uint8_t buffer_id, uint16_t start_page, uint16_t page_num, uint16_t *found_id)

Busca o template de um CharBuffer indicado por [buffer_id](#) (1 ou 2).

[start_page](#): Posição inicial do banco (normalmente 0).

[page_num](#): Número de posições, a partir de [start_page](#), que devem ser verificadas.

[found_id](#): Ponteiro de saída — recebe o ID/posição do template quando a busca tem sucesso.

Retorno: K044_OK se encontrado, se não K044_ERR_FP

k044_fp_reg_model(void)

Mescla os templates armazenados nos CharBuffers 1 e 2 em um único modelo, para ser gravado no banco com k044_fp_store(). O modelo mesclado fica disponível em ambas as posições.

Retorno: K044_OK ou erro

k044_fp_store(uint8_t buffer_id, uint16_t page_id)

Grava o modelo carregado no CharBuffer indicado por [buffer_id](#) (1 ou 2)

[page_id](#): Posição ID no banco onde gravar (0-999).

Retorno: K044_OK ou erro

k044_fp_load_char(uint8_t buffer_id, uint16_t page_id)

Carrega um modelo do banco em CharBuffer indicado por [buffer_id](#) (1 ou 2).

[page_id](#): Posição ID do banco (0-999).

Retorno: K044_OK ou erro

k044_fp_delete_char(uint16_t page_id, uint16_t count)

Remove um ou mais templates do banco de dados.

[page_id](#): Posição inicial (0-999).

[count](#): Número de registros para remover.

Retorno: K044_OK ou erro

k044_fp_empty(void)

Limpa todo o banco de dados de templates do modulo — Não pode ser desfeito.

Retorno: K044_OK ou erro

k044_fp_valid_template_count(uint16_t *count)

Conta o número de templates validos que existem no banco de dados.

count: Ponteiro de saída.

Retorno: K044_OK ou erro

k044_fp_led_off_fast(void)

Ajusta o LED do sensor para o estado de flashing - apagado (cor azul).

Use k044_fp_led_config() quando precisar de outra cor.

Retorno: K044_OK ou erro (cada uma)

k044_fp_led_on_fast(void)

Ajusta o LED do sensor para o estado de flashing - acesso (cor azul).

Retorno: K044_OK ou erro (cada uma)

k044_fp_led_flashing_fast(void)

Ajusta o LED do sensor para o estado de flashing - piscando (cor azul).

Retorno: K044_OK ou erro (cada uma)

k044_fp_led_breathing_fast(void)

Ajusta o LED do sensor para o estado de flashing - respirando (cor azul).

Retorno: K044_OK ou erro (cada uma)

k044_fp_read_nfpag(k044_fp_nfpag_t *page)

Le a pagina de informações do modulo.

page: Ponteiro de saída. NULL se quiser apenas confirmar se o modulo responde.

Retorno: K044_OK ou erro

k044_fp_sleep(void)

Coloca o modulo em modo de baixo consumo.

Qualquer rotina k044_fp_* chamada depois, desperta o modulo automaticamente.

Retorno: K044_OK ou erro

k044_fp_led_config(uint8_t func, uint8_t start_color, uint8_t end_color, uint8_t cycle_times)

Controla o LED do sensor com controle total sobre modo, cores e número de ciclos.
(diferente das rotinas k044_fp_led_*_fast(), que são pré-fixadas em azul).

func: Configura o modo de iluminação do Led.

CONSTANTE (FUNC)	VALOR	EFEITO
K044_FP_LED_BREATHING	1	Respiração
K044_FP_LED_FLASHING	2	Piscando
K044_FP_LED_ALWAYS_ON	3	Sempre aceso
K044_FP_LED_ALWAYS_OFF	4	Sempre apagado
K044_FP_LED_GRADUAL_ON	5	Acende gradualmente
K044_FP_LED_GRADUAL_OFF	6	Apaga gradualmente

start_color: cor inicial — bitmask de 3 bits (bit0=azul, bit1=verde, bit2=vermelho):

CONSTANTE	VALOR	BITS (A/V/VM)
K044_FP_LED_COLOR_BLUE	1	001 — azul
K044_FP_LED_COLOR_GREEN	2	010 — verde
K044_FP_LED_COLOR_CYAN	3	011 — azul + verde
K044_FP_LED_COLOR_RED	4	100 — vermelho
K044_FP_LED_COLOR_MAGENTA	5	101 — azul + vermelho
K044_FP_LED_COLOR_YELLOW	6	110 — verde + vermelho
K044_FP_LED_COLOR_WHITE	7	111 — todas

end_color: cor final — use a mesma de **start_color** para uma cor solida, ou uma diferente para fazer uma transicao entre as duas cores.

cycle_times: numero de ciclos do efeito

0 = infinito, ate o próximo comando de LED).

Retorno: K044_OK, ou K044_ERR_FP se o sensor rejeitar o comando.

k044_fp_auto_identify(uint8_t score_level, int16_t id_number, uint16_t *found_id, uint16_t *score)

Identificação automática com [captura](#), [criação do template](#) e [busca](#) em um único comando.

[score_level](#): 1-9 (recomendado 5). — 1 (mais permissivo) a 9 (mais rigoroso)

[id_number](#): -1 — busca em todo o banco de dados ou um ID específico (0-999)

[found_id](#): Ponteiro de saída — recebe o ID encontrado.

[score](#): Ponteiro de saída — recebe a pontuação de confiança do match.

Retorno: K044_OK se identificado, se não K044_ERR_FP.

k044_fp_auto_enroll(uint16_t id, uint8_t entries)

Cadastro automático multi-etapa guiado pelo módulo.

[id](#): 1-999 — identificador onde o cadastro será gravado (1-999).

[entries](#): 1-3. — Número de amostras do dedo a realizar (1-3, recomendado 3, para maior precisão).

Prefira [k044_fp_enroll\(\)](#) como alternativa mais previsível — ela reimplementa o mesmo fluxo chamando as rotinas básicas diretamente, sem depender do AutoEnroll interno do firmware.

Retorno: K044_OK ou erro

k044_fp_set_callback(k044_fp_callback_t cb, void *userdata)

Registra uma função de callback para acompanhar, em tempo real, o progresso de uma operação mais longa do fingerprint (cadastro ou busca).

— Utilizado para dar feedback ao usuário (ex.: "coloque o dedo", "capturado, retire o dedo", "gravando...")

[cb](#): ponteiro para uma função com a assinatura `void (*)(int status, int step, void *userdata)`.

[NULL](#) desregistra o callback atual.

Os dois primeiros parâmetros recebidos por essa função, significam:

[status](#) — 0x00 quando o passo correspondente teve sucesso;

qualquer outro valor é um código de confirmação do sensor

(o mesmo tipo de código devolvido por [k044_fp_last_confirmation_code\(\)](#))

— passe esse valor para [k044_fp_confirmation_str\(\)](#) para obter uma descrição textual do que aconteceu.

[step](#) — o passo atual da operação em andamento.

O significado depende de qual função disparou o callback:

* [k044_fp_enroll\(\)](#) e [k044_fp_search_retry\(\)](#)

Essas rotinas utilizam as constantes

K044_FP_STEP_WAIT_FINGER	1
K044_FP_STEP_CAPTURED	2
K044_FP_STEP_REMOVE_FINGER	3
K044_FP_STEP_MERGING	4
K044_FP_STEP_STORING	5

* [k044_fp_auto_enroll\(\)](#) (processo interno do próprio firmware):

Utiliza uma numeração própria do módulo:

0 = captura de imagem,

1 = extração de característica,

2 = gravação,

Etc.

— Não correspondente as constantes K044_FP_STEP_* acima.

[userdata](#): Ponteiro opaco definido e repassado sem alteração a cada chamada do callback

— Utilize para identificar o contexto ou objeto da aplicação que disparou a operação).

Retorno: void

k044_fp_enroll(uint16_t id, int finger_timeout_ms)

Cadastro manual em 3 leituras ([GetImage+GenChar+RegModel+Store](#))

Alternativa mais previsível ao **AutoEnroll** ([k044_fp_auto_enroll\(\)](#)).

Reporta o progresso via callback, se registrado ([k044_fp_set_callback\(\)](#)).

[id](#): identificador onde o cadastro sera gravado (1-999).

[finger_timeout_ms](#): tempo máximo de espera pelo dedo, em milissegundos, em cada uma das 3 capturas — se o tempo esgotar sem detectar o dedo, a função retorna `K044_ERR_FP_TIMEOUT`.

Retorno: `K044_OK`, ou `K044_ERR_FP/K044_ERR_FP_TIMEOUT`

k044_fp_search_retry(uint8_t buffer_id, uint16_t start_page, uint16_t page_num, int finger_timeout_ms, int max_attempts, uint16_t *found_id)

Identificação com repetição automática: Captura e busca no banco, e tenta novamente em caso de falha, até [max_attempts](#) tentativas.

[buffer_id](#): qual CharBuffer usar internamente para gerar o template de cada tentativa — 1 ou 2.

[start_page](#): Posição inicial do banco a partir de onde a busca começa.

[page_num](#): Quantas posições, a partir de [start_page](#), devem ser varridas em cada tentativa de busca.

[finger_timeout_ms](#): Tempo máximo de espera pelo dedo, em milissegundos, em cada tentativa.

[max_attempts](#): número máximo de tentativas de **captura+busca** antes de desistir.

[found_id](#): ponteiro de saída — recebe o ID encontrado, quando executado com sucesso.

Retorno: `K044_OK` se encontrou, se não `K044_ERR_FP/K044_ERR_FP_TIMEOUT`

k044_fp_confirmation_str(int code)

Traduz um código de confirmação do modulo em uma descrição textual legível.

[code](#): Código a traduzir (0-53)

— Normalmente o valor recebido no parâmetro status de um callback ([k044_fp_set_callback\(\)](#)) ou devolvido pela própria [k044_fp_last_confirmation_code\(\)](#).

Retorno: `const char*` (string estática, não precisa ser liberada)

k044_fp_last_confirmation_code(void)

Retorna o último código de confirmação recebido de qualquer chamada `k044_fp_*`

— Use para detalhar uma falha reportada como `K044_ERR_FP`, passando o valor retornado para [k044_fp_confirmation_str\(\)](#).

Retorno: `int` (código, ou `0xFF` se a resposta não pode ser interpretada)

k044_fp_raw_test(const uint8_t *data, uint16_t data_len, uint8_t *resp, uint16_t *resp_len, uint16_t max_resp)

Diagnostico de baixo nível: **NÃO usar para comunicação normal com o modulo.**

Usado apenas para testar o sensor isoladamente, enviando um pacote bruto e observando a resposta crua.

[data](#): buffer com os bytes a enviar diretamente ao sensor.

[data_len](#): quantidade de bytes em data a enviar.

[resp](#): buffer de saída onde a resposta do sensor é copiada.

[resp_len](#): ponteiro de saída — recebe a quantidade real de bytes recebidos na resposta.

[max_resp](#): capacidade do buffer resp, em bytes (limite máximo que pode ser copiado para ele).

Retorno: `K044_OK` se o comando foi aceito e a transação completou.

12. Termo de Serviço

A Avanttec Tecnologia assegura como fabricante, a garantia contra defeitos de fabricação pelo prazo informado na etiqueta lacre localizado no produto. O prazo de garantia é contado a partir da data de emissão da Nota Fiscal de venda, conforme especificado neste Termo e na lei.

A Avanttec Tecnologia não deve ser responsabilizada por erros e/ou omissões técnicas ou editoriais aqui contidas, nem por danos incidentais ou consequentes resultantes do fornecimento, desempenho ou uso do produto.

A Avanttec Tecnologia garante que os produtos cobertos por este Termo estão livres de defeitos em seus componentes eletrônicos, mecânicos e de defeitos de montagem, durante o período da Garantia Legal, uma vez respeitado as condições normais de uso.

A Avanttec Tecnologia assegura que durante o período e nas condições da Garantia, os módulos ou as peças defeituosas serão substituídos por peças novas, de qualidade e tecnologia equivalente ou superior, respeitando as especificações originais do produto e a legislação brasileira aplicável, sendo que todas as partes defeituosas e substituídas na vigência da Garantia Legal passarão a ser de nossa propriedade para serem descartadas de modo adequado, conforme nosso Programa de Responsabilidade Ambiental.

13. Termo de Assistência Técnica

No caso de constatação de defeito do produto, o cliente deverá entrar em contato com o serviço de suporte ao cliente para devida orientação sendo de exclusiva responsabilidade do cliente o envio do produto para a assistência técnica localizada na:

Rua Poruborás, 61 - Vila Conceição – Diadema/SP- CEP: 09990-340.

É necessário que as mercadorias enviadas acompanhadas dos documentos fiscais descritos abaixo:

Dados Cadastrais

Avanttec Máquinas e Equipamentos Industriais eireli

CNPJ: 30.472.190/0001-00

IE: 286.497.141.110

Endereço: Rua Poruborás, 61 - Vila Conceição – Diadema/SP- CEP: 09990-340

E-mail: suporte@avanttectecnologia.com.br

Telefones: +55 (11) 4051-2781

Natureza da Operação

CFOP: 5.915- Remessas para Conserto (Dentro do estado);

CFOP: 6.915- Remessas para Conserto (Fora do estado).

Descrição do Defeito Apresentado

Enviar junto com os equipamentos, laudo descrevendo o problema que o equipamento apresenta e informação do número de Nota Fiscal de compra e número de série do produto (localizado em etiqueta embaixo do produto). É necessário o envio do arquivo XML da Nota Fiscal para email suporte@avanttectecnologia.com.br.

O presente Termo de Garantia, não terá validade nos casos de:

1. Ligação incorreta de periféricos externos que possam danificar o produto;
2. Partes consumíveis, tais como baterias não recarregáveis, etc.;
3. Danos causados por violação do produto, tentativa de reparo ou ajuste por terceiros não autorizados pela Avanttec;
4. Limpeza, reparos cosméticos ou desgaste resultante do uso natural do produto;
5. Danos causados por mau uso, abuso, queda, negligência, imprudência ou imperícia;
6. Danos causados por equipamentos que produzam ou induzam interferências eletromagnéticas;
7. Danos causados por problemas de instalação elétrica ou instalações em desacordo com as normas ABNT;
8. Danos causados por modificações e alterações no produto original;
9. Danos causados por agentes da natureza (descargas elétricas, inundações, incêndios, desabamentos, terremoto, etc.);
10. Utilização de peça, parte, componente não original no produto;
11. Dano causado no rompimento, ausência, rasura ou adulteração do número de série do produto e lacre de garantia;
12. Abertura do produto em qualquer de suas partes externas ou internas;
13. Qualquer modificação do produto ou atualização sem autorização prévia da Avanttec;